

DFS

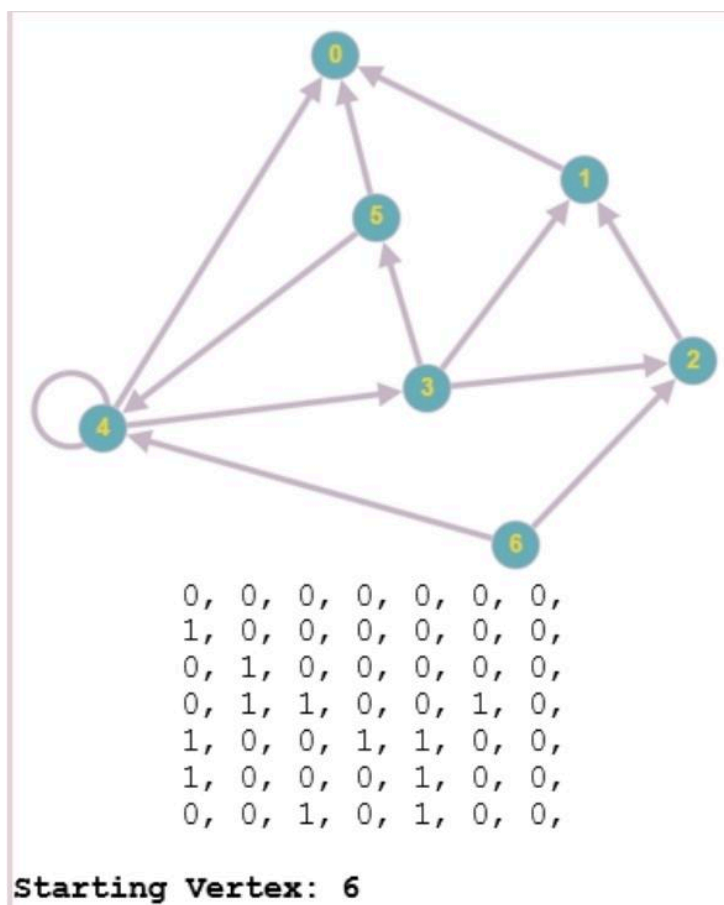
DFS

Theory:

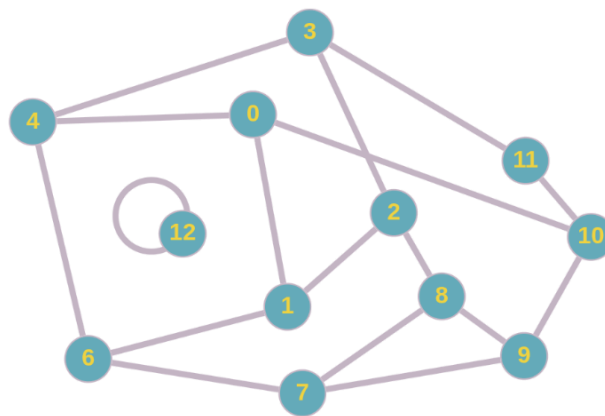
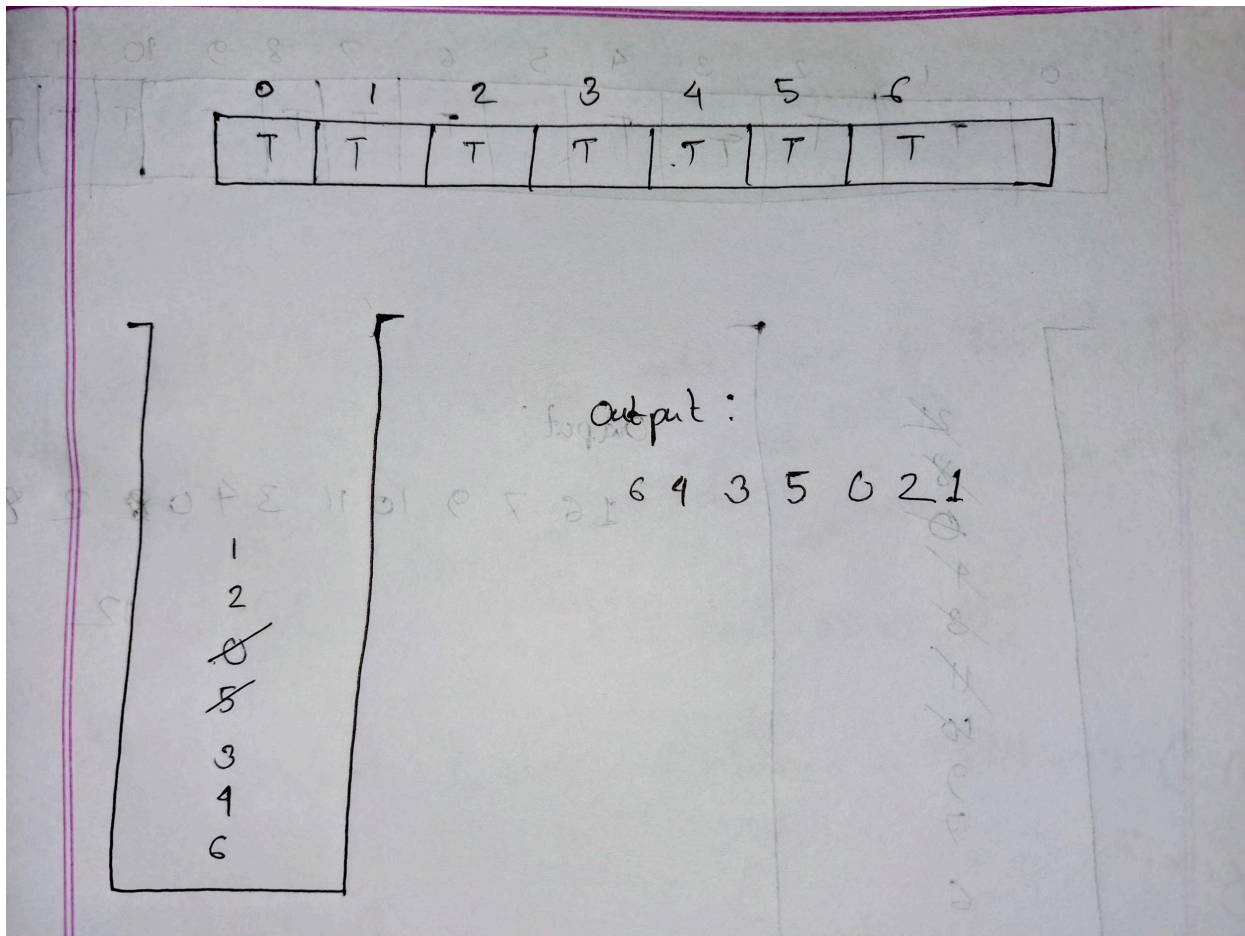
- DFS uses **Recursion(Backtracks)**
- Used for finding path or solving maze
- Discovery time and starting time possible

Process:

- Uses boolean array and stack
- Takes only one adjacent vertex
- When pushed
 - 1. Make the boolean index True[visited marked]
 - 2. Print Output
- Start popping and backtracking when all adjacent vertex have boolean index true

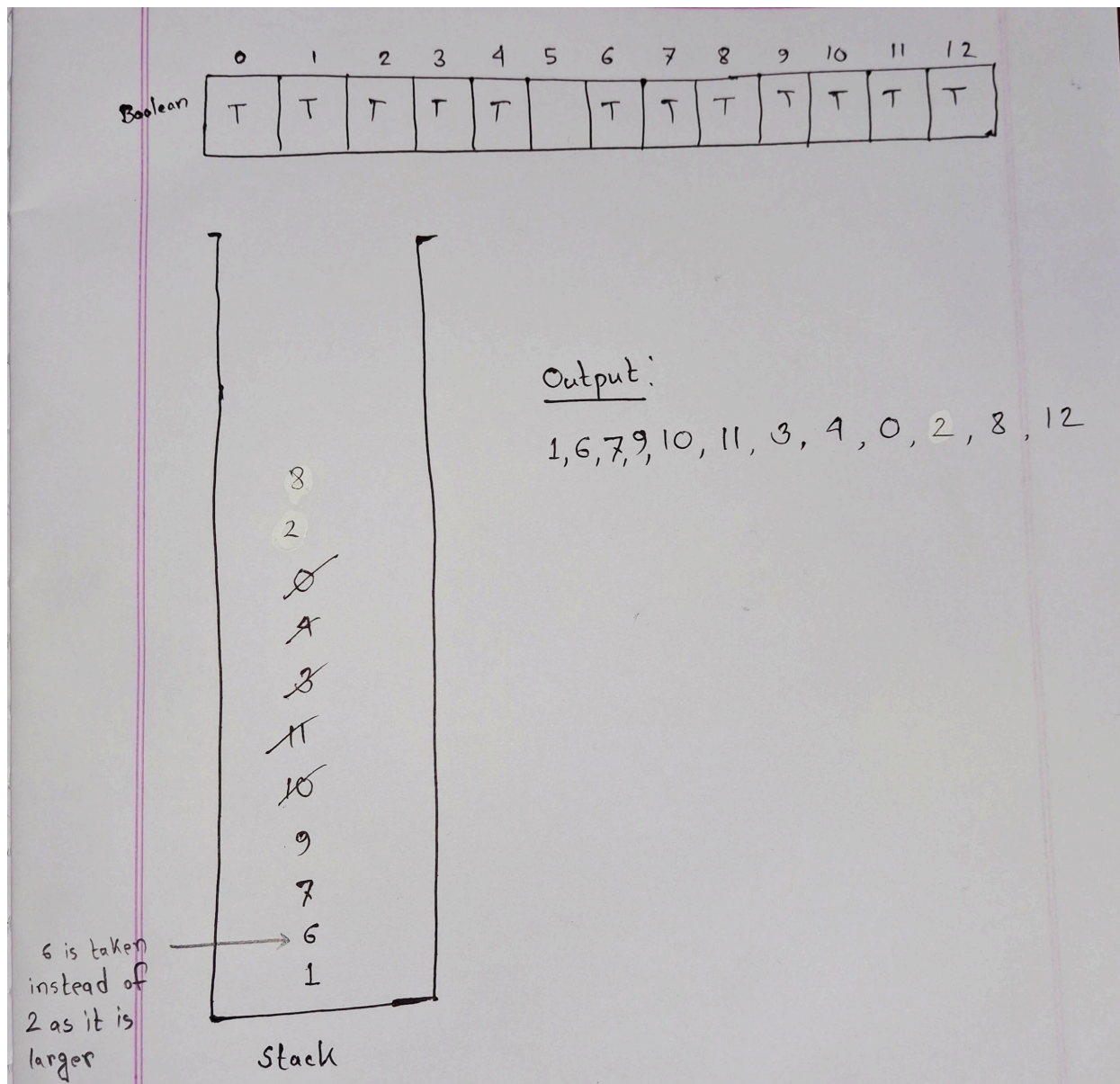


Solve in descending order(larger node first)



Find the DFS traversal of this graph. Consider 1 as the starting point.
Take the largest vertex first.

Solve in descending order (larger node first)

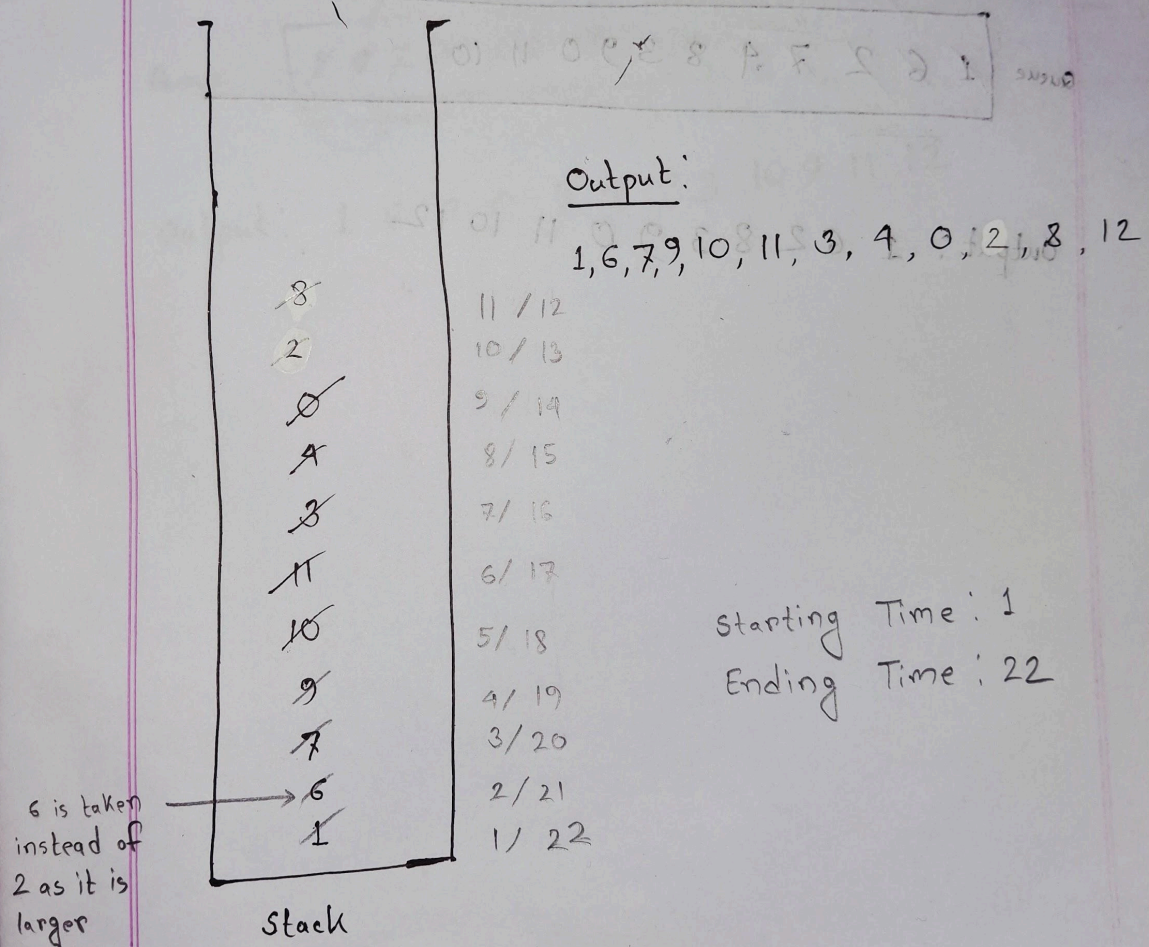


Find the discovery time/starting time and finishing time

Graph Theory 5 Extra: Discovery and Finishing time using DFS (Bangla | বাংলা)

Boolean

0	1	2	3	4	5	6	7	8	9	10	11	12
T	T	T	T	T	T	T	T	T	T	T	T	T



Starting Time : 1
Ending Time : 22

BFS

BFS

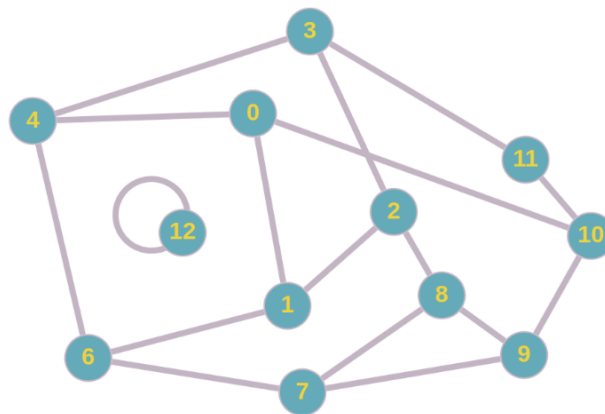
Theory

- Shortest path in unweighted graphs
- Level-order traversal of trees
- Network broadcasting (spreading info layer by layer)
- Finding connected components in graphs
- Discovery/Starting time and finishing time isn't possible in BFS
- Uses Queue

Process:

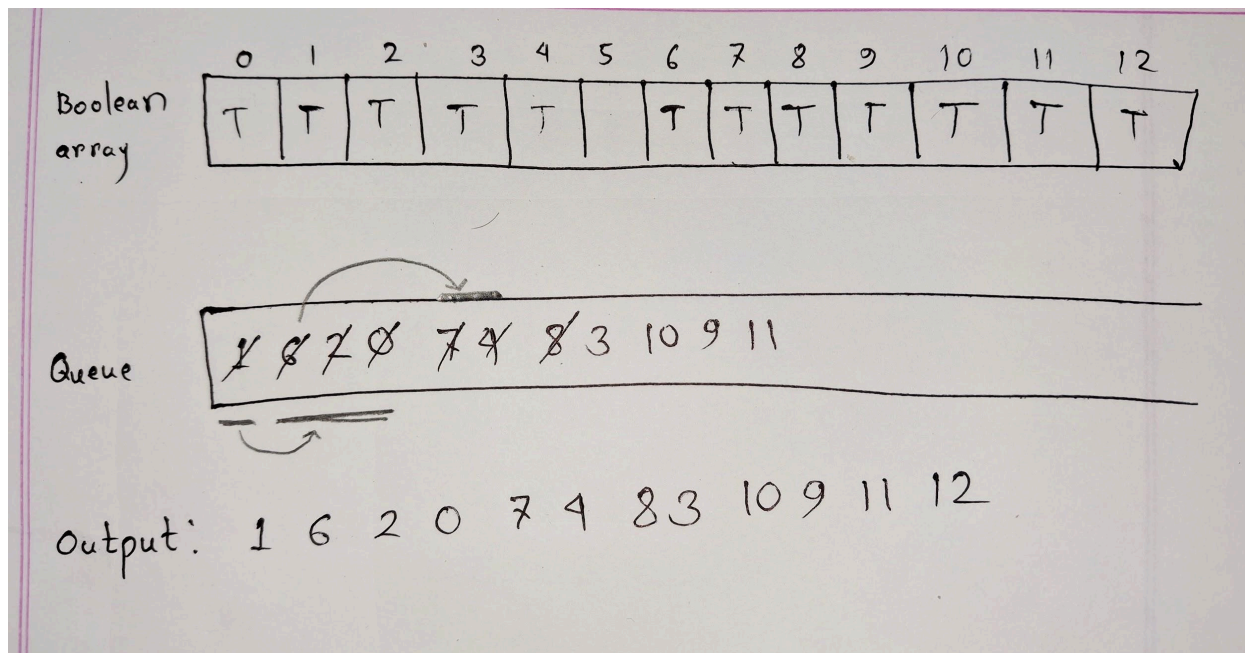
- Uses boolean array and Queue
 - Takes only all adjacent vertex
 - When peeked
 - 1. Make the boolean index True[visited marked]
 - 2. Print Output
 - Start dequeue when all adjacent vertex have boolean index true
-

1.



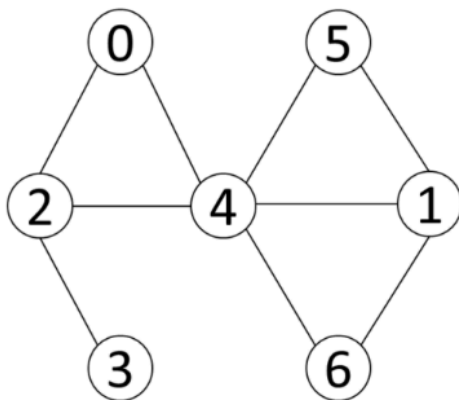
Simulate the BFS algorithm using a **Queue** choosing any specific vertex as a starting point.

Solve in descending order(larger node first)



2.

i. Which one of the following sequences is a possible order of visiting the nodes in the graph below if we use BFS traversal?



- A. 0, 5, 2, 4, 1, 3, 6
- B. 4, 6, 1, 5, 3, 2, 0
- C. 0, 4, 5, 1, 6, 2, 3
- D. 1, 4, 5, 6, 0, 2, 3

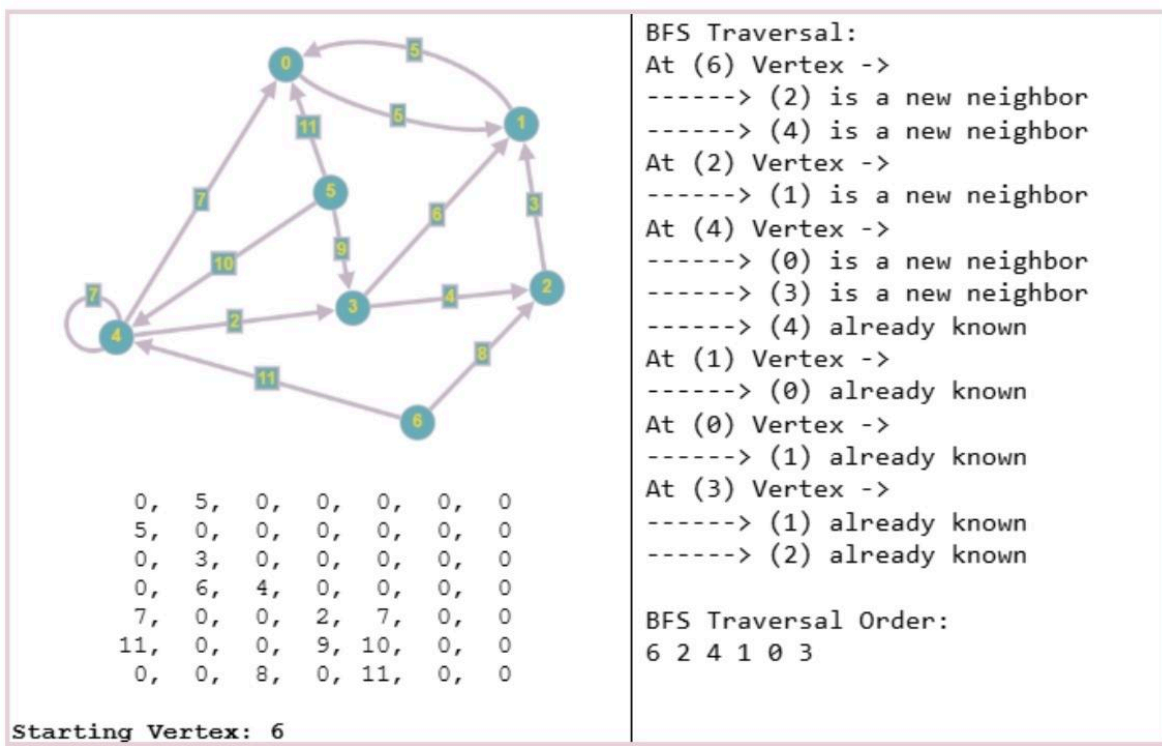
Answer: D

3. Breadth-First Search (BFS) can always visit all vertices of a directed graph in a single execution.

[False]

Correct answer: BFS can visit all vertices only if all vertices are reachable from the chosen source vertex. In a directed graph, some vertices may be unreachable.

4.



Solve in ascending order(smaller node first)

0	1	2	3	4	5	6
T	T	T	T	T		T

~~2 4 1 0 3~~

Output: 6 2 4 1 0 3

Tab 3

Time & Space complexities

→ Heap:

- * Max-heapify / Sink : $O(\log n)$
- * Heap Increase Key / Swim : $O(\log n)$; Best : $O(1)$
- * Heap-insert : $O(\log n)$
- * Extract : $O(\log n)$
- * Get max/min : $O(1)$
- * Build heap : $O(n)$
- * Heap sort : $O(n \log n)$

→ ^{Search} Binary Tree:

- * Fast searching : $O(\log n)$ (if balanced)
- * Insertion & Deletion : $O(\log n)$ ("")

→ For unbalanced there are : $O(n)$

→ Graph:

List Space Complexity: → List : $O(N+M)$
→ Matrix : $O(N^2)$

Whether an edge exists: → List : $O(\deg(u))$
→ Matrix : $O(1)$